

[Learning Spectral Point Distributions with Neural Energy Fields](#)

[Executive Summary](#)

[1. Research Objective](#)

[2. Positioning Relative to Prior Work](#)

[3. Proposed Contributions](#)

[4. Overall Pipeline](#)

[5. Staged Implementation Plan](#)

[6. Computational Budget & Training Strategy](#)

[7. Experimental Matrix](#)

[8. Required Ablation Studies](#)

[9. Evaluation Protocol](#)

[10. Applications](#)

[11. Theoretical Analysis](#)

[12. Risk Analysis & Mitigations](#)

[13. Timeline & Resourcing](#)

[14. Implementation Plan](#)

[15. Recommended Initial Hyperparameters](#)

[16. What Not to Do Initially](#)

[17. Publication Positioning](#)

[18. Central Scientific Claim to Test](#)

[19. Immediate Next Step](#)

[20. Success Criteria](#)

Learning Spectral Point Distributions with Neural Energy Fields

A standalone research proposal — v3, detailed Domain: Computer Graphics — Procedural Point Distributions, Sampling, Neural Energy Models

Executive Summary

The core system is a dynamical system whose *only* driver of particle motion is a learned energy field: a small MLP takes pairwise distance (and a target-spectrum condition) as input, its gradient gives a force, and particles are updated under that force alone — no hand-designed repulsion, attraction, or phasor term. The network is trained by backpropagating a spectral-matching loss through the entire unrolled simulation.

This is simple as a concept, but not a foregone conclusion as a result. The central open question, flagged throughout this document, is whether a spectral-matching loss alone is *sufficient* to produce a well-spaced point distribution, or whether it has a structural blind spot that requires an explicit spatial-regularity loss to close. Section 6 works through why that blind spot exists and how to close it. The recommended path is to start with the minimum viable loss (spectral + slope + one spacing term), validate the two non-negotiable gradient checks before training anything, and only add further metrics (anisotropy, mean spacing, discrepancy) reactively, in response to a diagnosed failure — not preemptively.

1. Research Objective

Develop a neural energy-based particle system that synthesizes 2-D point distributions with a user-specified frequency-domain spectrum, generalizing continuously across the blue/pink/red/white/violet noise family rather than requiring a separately hand-tuned or separately optimized system per noise color.

Primary hypothesis: A neural pairwise interaction energy, embedded inside a physically-motivated particle dynamical system (positions evolving under the negative gradient of a learned energy), can learn to reproduce and interpolate across an entire family of target power spectra — while remaining interpretable, provably stable in the conservative case, and cheap to evaluate at inference time compared to per-target classical optimization.

This is a standalone system: it does not depend on, extend, or preserve any pre-existing hand-designed dynamical system. Any short-range regularization needed to prevent particle collapse is either learned by the network itself or included as a minimal, clearly-labeled numerical stabilizer.

2. Positioning Relative to Prior Work

Three adjacent bodies of work need to be addressed before implementation begins, since each offers a plausible alternative solution to the same problem.

2.1 Classical continuous spectral control

Spectral/Fourier-domain optimization of blue-noise and $1/f^\beta$ point sets, Lloyd-relaxation and capacity-constrained variants, and direct particle-position optimization against a log-PSD loss can already hit arbitrary target slopes without any learned component. This is the most important comparison in the whole proposal: if a classical per-target optimizer matches the neural model’s quality at comparable cost, “continuous control” alone is not a sufficient contribution.

The differentiating claim must be one of: - Generalization across spectra without per-target re-optimization (train once, infer instantly for any new γ) - A provable stability guarantee that ad hoc classical optimizers lack (Section 11) - A small, interpretable model versus an iterative per-target solve

Pick one as the headline claim early — it shapes what gets emphasized in the writeup and which ablations matter most.

2.2 Learned point-process and blue-noise generation

GAN- and diffusion-based point/sample generators and learned optimal-transport samplers already produce blue-noise-like distributions. This proposal’s system should be distinguished on mechanism — an interpretable energy embedded in a dynamical system, not a black-box generator — and that mechanism should be shown to *buy something*: controllability, the stability argument in Section 11, and a smaller model, rather than being justified on raw sample quality alone.

2.3 Neural interaction-energy / neural force fields

Learned pairwise energies and neural force fields have precedent in particle simulation and molecular dynamics (permutation-invariant pairwise energies, energy-conserving neural ODEs). This lineage should be cited to strengthen Stage 4–5 below; the novelty is the application to spectral point synthesis and conditioning on a target spectrum, not the energy-gradient force construction itself.

Action item before Stage 1: run a structured literature search (suggested terms: “continuous power spectrum point distribution optimization,” “neural blue noise generation,” “learned point process sampling,” “differentiable particle simulation neural force field,” “energy-based point set optimization graphics”) and produce a short related-work table: method, mechanism, spectral controllability, reported cost. This determines the comparison baseline target in Section 6.3.

3. Proposed Contributions

ID	Contribution	Description
C1	Conditional neural spectral energy	Learn an interaction energy conditioned on target spectral characteristics.
C2	Self-contained neural dynamical system	A particle system whose only interaction law is the learned energy (plus, if needed, a minimal fixed stabilizer).
C3	Differentiable spectral objective	Backpropagate from the final PSD through particle dynamics to train the interaction model.
C4	Continuous spectral control	Generate arbitrary/interpolated spectral exponents rather than only predefined noise colors.
C5	Learned regularity (stretch)	Test whether the network discovers repulsion-like spacing behavior purely from the spectral objective, without being given a hand-designed repulsion term.
C6	Anisotropic extension (stretch)	Control direction-dependent frequency distributions using 2-D target spectra.
C7	Spatially varying extension (stretch)	Allow desired spectral characteristics to change across the domain.

4. Overall Pipeline

```
Target spectrum  $\gamma \rightarrow$  condition encoder  $\rightarrow$  neural pairwise energy  $E_\theta(x_i, x_j, \gamma)$ 
     $\rightarrow$  neural force  $F_i = -\partial E_\theta / \partial x_i$ 
     $\rightarrow$  particle dynamics  $x_i(t+1) = x_i(t) + \Delta t \cdot F_i$ 
     $\rightarrow$  final point set  $\rightarrow$  rasterization  $\rightarrow$  FFT/PSD
     $\rightarrow$  spectral loss  $\rightarrow$  backpropagation  $\rightarrow$  update  $E_\theta$ 
```

The network is not a point-cloud generator. It learns an interaction law embedded inside a dynamical system, and the final point distribution is an equilibrium-like configuration produced by particle evolution under that law.

5. Staged Implementation Plan

Stage 0 — Reference Distributions and Baseline Metrics

Define reference/sanity-check distributions (Poisson-disk sampling, jittered grids, pure random) with known or well-characterized spectral properties. These serve as ground truth for validating the spectral analyzer in Stage 3, and as the “known spectra” row of the experimental matrix (Section 7). Record baseline runtime and peak GPU memory at $N=256$ and $N=512$ — this becomes the reference point for the compute budget in Section 6.

Stage 1 — Define the Initial Learning Problem

2-D square domain, $N=256$ or $N=512$. First target family: power-law spectrum $P^*(f) = C \cdot f^\gamma$, $\gamma \in$ approximately $[-2, +2]$, covering red, pink, white, blue, and violet-like regimes. Do not begin with arbitrary PSDs, anisotropy, or spatially varying targets.

Stage 2 — Differentiable Particle Representation

State $X = \{x_i\}$, each x_i a 2-D position. Implement the full simulation in PyTorch or JAX so gradients pass from the final spectral loss to the neural parameters. Use a simple fixed-step Euler or semi-implicit update first.

Stage 3 — Build and Validate the Spectral Analyzer

- Rasterize particle positions into a density image $D(x,y)$ using a **soft Gaussian splat kernel** (not hard/nearest-pixel splatting, which gives zero or noisy position gradients).
- Compute $\hat{D} = \text{FFT}(D)$, $P(u,v) = |\hat{D}(u,v)|^2$, radial frequency $f = \sqrt{u^2+v^2}$, radially average to $P(f)$, fit log-log slope to estimate $\hat{\gamma}$.
- Validation requirement:** verify the analyzer against Stage 0’s known reference distributions, and run a finite-difference gradient check on $\partial D / \partial x_i$ before any training begins. This must pass before Stage 4.

Stage 4 — Neural Pairwise Energy

For each particle pair: $r_{ij} = \|x_i - x_j\|$, $e_{ij} = \text{NN}_\theta(r_{ij}, \gamma)$, $E_\theta(X, \gamma) = \sum_{\{i<j\}} e_{ij}$. The sum over unordered pairs makes the formulation permutation-invariant. The network learns a distance-dependent interaction potential conditioned on the target spectrum. No hand-designed repulsion/attraction/phaser term is assumed present.

Architecture decision — free MLP vs. a built-in divergence prior. A fully unconstrained $\text{MLP}(r)$ has no structural guarantee against particles occupying the same point: unlike classical potentials ($1/r$, $1/r^2$) that are built to diverge as $r \rightarrow 0$, a plain MLP can output a small, finite, non-repulsive value at $r \approx 0$ if that minimizes the loss. Two options:

Approach	Trade-off
Fully free $\text{MLP}(r)$	Maximum flexibility; cleanest test of whether repulsion emerges purely from training pressure (contribution C5); no structural safety net against collapse
$\text{MLP}(r)$ + a minimal fixed term that diverges as $r \rightarrow 0$	Guarantees non-overlap by construction; reintroduces a small hand-designed piece, but a far smaller and more defensible one than a full repulsion/attraction/phaser system

Recommendation: start with the fully free MLP. Keep the divergence-prior version as a documented fallback, not a default — if collapse appears in the Stage 7 diagnostic check, switch to it and report the comparison as a finding.

Stage 5 — Convert Energy to Neural Force

Predict energy rather than directly predicting forces: $F_i^{\wedge \text{NN}} = -\partial E_\theta / \partial x_i$, via automatic differentiation. This keeps the learned force field consistent with the learned energy and preserves the stability argument in Section 11.

Stage 6 — Particle Dynamics

$x_i(t+1) = x_i(t) + \Delta t \cdot F_i^{\wedge \text{NN}}$. If early experiments show particle collapse or degenerate clustering under the spectral loss alone, add a minimal, fixed, clearly-labeled stabilizing repulsion term (e.g., a short-range inverse-square push) — but treat this as a fallback to test *against*, not a default. The first experiments should attempt training with **no** hand-designed term at all, to test whether spacing regularity emerges purely from the spectral + spacing loss (this is contribution C5 and a natural ablation).

Stage 7 — Spectral Training Objective

After T simulation steps, measure $P_\theta(f)$ and compare to $P^*(f)$: $-L_{\text{spec}} = \text{mean}_f [\log P_\theta(f) - \log P^*(f)]^2$ - $L_\gamma = (\hat{\gamma} - \gamma)^2$ - $L_{\text{spacing}} =$

CV_NND(X,T) — coefficient of variation of nearest-neighbor distances. Chosen as the single primary spatial-quality term because it is cheap, differentiable, and directly interpretable (lower CV = more regular spacing). The pair-correlation function g(r) is reserved as an evaluation-time metric (Section 9), not a training loss, since it is harder to differentiate through reliably.

Full objective (minimum viable version): $L = \lambda_s \cdot L_{\text{spec}} + \lambda_\gamma \cdot L_\gamma + \lambda_q \cdot L_{\text{spacing}}$

Why L_{spacing} is structurally necessary, not a nice-to-have. The spectral loss is measured through a rasterized density image, produced by splatting particles with a Gaussian kernel of width σ . That kernel acts as a low-pass filter: any structure finer than σ — e.g. two particles collapsed almost on top of each other — is smeared into what looks like a single blob and is invisible to the FFT. This means the network can, in principle, satisfy L_{spec} by letting particles collapse into tight sub- σ clusters that are spaced apart at a coarser scale, producing a radial power spectrum that looks correct at the frequencies the rasterizer can resolve while being spatially degenerate. L_{spec} cannot see this failure mode by construction — it is a blind spot in the measurement, not a training difficulty. L_{spacing} , computed directly on raw particle positions rather than through the rasterizer, is what closes this loophole, and should be treated as required from the first experiment rather than added later if problems appear.

Diagnostic check (required at every milestone): after training, plot the pair-correlation function g(r) at small r. A spike near r=0 despite a low spectral loss is the signature of sub-kernel collapse — confirmed directly, rather than inferred. This check determines whether σ needs to shrink, whether λ_q needs to increase, or whether the divergence-prior fallback architecture (Stage 4) is needed.

Stage 7a — Optional Additional Metrics (add reactively, not by default)

The following can extend the loss or evaluation protocol, but should be added only in response to a specific diagnosed failure at Stage 9’s milestone, not folded in preemptively. Each additional loss term requires its own weight and increases the risk of competing gradients slowing or destabilizing training — the recommended default is the three-term loss above, kept minimal until it demonstrably needs more.

Metric	Role	Why
Mean nearest-neighbor spacing	Optional training loss: $L_{\text{mean}} = (\text{mean}(\text{NND}) - \text{target_spacing})^2$	Complements CV_NND (which controls <i>relative</i> regularity) by controlling absolute density/spacing when N and domain size don’t already fix it. Cheap and differentiable.
Anisotropy penalty	Optional training loss: angular variance of the 2-D PSD at each radius, penalized toward zero	Useful if isotropy should be actively enforced rather than hoped for during Stages 1–11; becomes the natural “target anisotropy = 0” case of the Stage 12 machinery, so implementing it early is not wasted work.
Discrepancy (e.g. star discrepancy)	Evaluation-only metric (Section 9), not a training loss	Expensive to compute (typically requires search over box subsets or a fine grid) with no clean cheap gradient; mostly tracks what $L_{\text{spec}} + L_{\text{spacing}}$ already push toward. Use to validate the final result, not to optimize directly.
Nyquist frequency	Sanity check on the Stage 3 analyzer setup, not a target to optimize	This is a property of the <i>measurement pipeline</i> (grid resolution and particle density), not of the point distribution. Before trusting L_{spec} , verify the target spectrum’s relevant features (e.g. blue noise’s characteristic low-frequency dip) fall well inside the frequency range the rasterization grid can actually resolve — otherwise training is fit against an under-resolved measurement.

Stage 8 — End-to-End Training

Sample γ uniformly from the training interval → generate X_0 → run T steps → rasterize → FFT/PSD → compute losses → backpropagate through the simulation → update θ with Adam → repeat with new initial distributions and target γ values.

Stage 9 — First Experimental Milestone

M1: with N=256 in a unit square, reliably generate a distribution whose measured spectral slope approaches any target $\gamma \in [-2,+2]$. Train only $\gamma=+1$ first, verify the loss decreases and the spectrum moves toward target, then test $\gamma = -1, 0, +1$, then continuous random γ .

Stage 10 — Continuous Spectral Interpolation

Train on $\gamma \in \{-2,-1,0,+1,+2\}$, test unseen values such as -1.5, -0.7, 0.3, 0.8, +1.5. This is the critical test of continuous learning versus category memorization, and the core evidence for contribution C4.

Stage 11 — Local Neural Energy for Scalability

Full pairwise energy is $O(N^2)$. After the first proof of concept, switch to k-nearest-neighbor interactions: $E_\theta = \sum_i \sum_{\{j \in N_i\}} e_\theta(x_i - x_j, \gamma)$. Test k = 16, 32, 64. Target: approximately $O(Nk)$ computation while preserving spectral quality.

Stage 12 — Anisotropic Extension (stretch)

Condition $c = [\gamma, \alpha, \theta]$, where α controls anisotropy strength and θ controls orientation. Use the relative vector rather than only distance: $e_{ij} = \text{NN}_{\theta}(\Delta x_{ij}, \gamma, \alpha, \theta)$. $L_{2D} = \text{mean}_{\{(u,v)\}} [\log P_{\theta}(u,v) - \log P^*(u,v)]^2$.

Stage 13 — Spatially Varying Extension (stretch)

$\gamma = \gamma(x,y)$; each particle receives $c_i = [\gamma(x_i), \rho(x_i), \alpha(x_i), \theta(x_i)]$; interaction depends on both particles: $e_{ij} = \text{NN}_{\theta}(\Delta x_{ij}, c_i, c_j)$. Strong demo: a single domain with a smooth transition from red/pink-like to white to blue-like behavior.

6. Computational Budget & Training Strategy

Stage 11 identifies $O(N^2)$ inference cost as a scalability concern, but the more immediate issue is training cost: Stage 8 backpropagates through $T=50\text{--}100$ unrolled steps, each with an $O(N^2)$ pairwise energy evaluation, for every training sample. Size this before Stage 4, not after hitting a wall.

6.1 Memory: truncated backpropagation through time

Full backprop through 50–100 steps with an autograd graph over all pairwise interactions at $N=256\text{--}512$ is likely memory-heavy on a single GPU. Plan for either **gradient checkpointing** (recompute forward activations during the backward pass) or **truncated BPTT** (backpropagate through a sliding window of the last k steps, run the rest under `no_grad`). State the default choice for Stage 8, and record peak memory at $N=256$ and $N=512$ as part of Stage 0’s baseline metrics.

6.2 Compute estimate

Before Stage 8, produce a rough GPU-hour estimate for milestone M1 (single γ , $N=256$) versus continuous- γ training ($N=256\text{--}512$) versus the local-energy milestone ($N=4096$). This drives whether Stage 11’s local-energy model should be pulled earlier if the full pairwise version is too slow to iterate on.

6.3 Comparison Baseline (required)

Add a non-neural continuous-spectrum baseline to the experimental matrix: a classical per-target optimizer — gradient-based or relaxation-based direct particle-position optimization against the log-PSD loss, run separately for each test γ . Compare against the neural model on:

- a. final spectral/spatial quality at matched γ
- b. wall-clock cost per new target γ — the classical baseline pays this cost per target; the neural model pays it once at training time and should be near-instant at inference

This is what turns “continuous spectral control” into a quantified claim rather than an assertion, and — given there is no Paper 1 to fall back on for credibility — this comparison now carries more of the paper’s weight than it would in an extension paper.

6.4 Differentiability Audit (required before Stage 4)

Operation	Why it breaks gradients	Fix
Domain boundary handling (wrap/teleport or hard clamp)	A particle crossing the edge jumps discontinuously; the gradient at that instant is undefined or misleading.	See Section 6.5 — minimum-image convention, soft boundary force, or straight-through estimator.
Neighbor selection for local interactions (Stage 11)	Which pairs are “close enough” is a discrete include/exclude decision.	Keep discrete top-k selection in the forward pass; make the interaction weight a smooth, distance-decaying function so gradients w.r.t. position are well-behaved for included pairs.
Rasterization for the PSD (Stage 3)	Hard nearest-pixel splatting gives ~zero gradient almost everywhere.	Soft Gaussian splat kernel, as specified in Stage 3.
Branching force rules (if distance < threshold, force A, else force B)	Creates a kink/jump in the force field at the threshold.	Replace with a smooth function that interpolates between regimes (e.g. sigmoid-weighted blend).
Fresh random sampling inside the simulation loop	Fine as an input; a problem if gradients need to flow through the sampling step itself.	Use a fixed noise draw per step (reparameterization) if noise must stay in the differentiable path.

6.5 Domain Boundary Fix

Choose based on whether periodicity is essential to the desired spectral behavior:

- Minimum-image convention** (preferred if periodicity matters): keep wrapping, but compute pairwise distances using the wrapped/shortest-path difference. Gradients pass through cleanly except at the measure-zero instant of an actual wrap event. Standard in periodic-boundary molecular dynamics.

- **Soft boundary force** (simpler, if periodicity is not essential): replace hard teleportation with a smooth repulsive force near the edge, keeping particles inside without wrapping. Removes the discontinuity entirely at the cost of true periodicity.
- **Straight-through estimator** (fastest patch, use with caution): exact wrap in the forward pass, identity in the backward pass. Introduces small, usually tolerable gradient noise; note explicitly as an approximation in ablations if used.

Required check: run the Stage 3 finite-difference gradient check specifically on a particle near the boundary, both before and after applying the fix. A passing check away from the boundary but failing near it is the signature of this exact problem.

7. Experimental Matrix

Experiment	Question	Primary metric
Known spectra	Can the model reproduce target slopes?	Slope / PSD error
Interpolation	Does it generalize to unseen γ ?	MAE of $\hat{\gamma}$
Initialization	Is the result independent of X_0 ?	PSD variance
Point count	Does it scale?	Quality vs. N / runtime
Ablation	Which component matters?	PSD + spatial metrics
Classical baseline	Does the neural model beat per-target classical optimization on cost or quality?	Quality-at-matched- γ and cost-per-new- γ
Anisotropy (stretch)	Can it reproduce 2-D spectra?	2-D spectral error
Spatial adaptation (stretch)	Can spectrum vary over space?	Local spectral error
Application	Does it improve a graphics task?	Task-specific metric

8. Required Ablation Studies

- Neural energy with no stabilizing term vs. with a minimal fixed repulsion term (tests contribution C5).
- Direct neural-force model versus neural-energy model (predicting force directly vs. predicting energy and differentiating).
- Spectral loss only versus spectral + spatial-quality (L_{spacing}) losses.
- Global pairwise energy versus local k-neighbor energy (Stage 11).
- Straight-through boundary estimator versus minimum-image convention, if the straight-through approximation is used anywhere in the final system.

9. Evaluation Protocol

PSD-MSE and slope error are necessary but not sufficient for a graphics venue; reviewers of sampling/stippling work expect spatial-statistics and, ideally, visual/perceptual evidence.

9.1 Spatial statistics (quantitative)

- Pair-correlation function $g(r)$, compared against the target family’s expected shape (evaluation-time only).
- Coefficient of variation of nearest-neighbor distances (training + evaluation).
- Discrepancy or Ripley’s K-function as a secondary, independent check of spatial regularity not directly optimized against.

9.2 Perceptual evaluation

For at least the stippling application (Section 10), render matched-density stipple/point images from (a) the neural model and (b) the classical per-target optimizer from Section 6.3, and collect pairwise preference or artifact-detection judgments from a modest pool of participants. An informal lab study is sufficient for a first submission.

9.3 Reproducibility

- Fix and report random seeds for all reported numbers (≥ 3 seeds per configuration).
- Release code and the trained condition-conditioned energy network alongside the paper.

10. Applications

- Adaptive stippling.** Map image content to density and spectral conditions and generate spatially adaptive point distributions.
- Monte Carlo sampling.** Investigate whether controllable spectral distributions reduce variance or improve convergence in rendering.
- Anisotropic object placement.** Place oriented objects, fibers, vegetation, or repeated assets using directional spectral control.

11. Theoretical Analysis

For the simplified conservative system $dX/dt = -\nabla E_\theta(X)$, the energy satisfies $dE/dt = -\|\nabla E_\theta\|^2 \leq 0$. This gives a stability interpretation for the learned energy component, and is a central piece of the differentiation argument in Section 2.2 — it is a guarantee classical per-target optimizers and black-box generators typically do not offer. The complete system includes discrete integration and possibly a stabilizing term, so global convergence should not be claimed without further proof; state this limitation explicitly.

12. Risk Analysis & Mitigations

Risk	Likelihood / Impact	Mitigation
Classical baseline matches neural quality at similar cost, weakening the core claim	Medium / High	Run Section 6.3 comparison early (right after M1), not at the end. If the gap is small, pivot the contribution toward generalization/anisotropy/spatial adaptation rather than raw continuous control.
Without any hand-designed repulsion, particles collapse or cluster degenerately	Medium / High	L_{spacing} (CV_NND) is included in the loss from Stage 7 onward; monitor CV_NND during training as an early-stopping/diagnostic signal; fall back to the minimal stabilizer (Stage 6) if collapse persists, and report this as a finding rather than hiding it.
Backprop-through-time is too slow/memory-heavy to iterate at $N=256\text{--}512$, $T=50\text{--}100$	Medium / High	Adopt truncated BPTT or checkpointing by default (Section 6.1); pull Stage 11’s local k-NN energy earlier if needed.
Rasterization gradients are noisy or biased, causing the network to fit measurement artifacts	Medium / High	Mandatory finite-difference gradient check on the analyzer before Stage 4 (Section 5, Stage 3).
Anisotropic/spatial extensions (Stages 12–13) do not converge within the project timeline	Medium / Medium	Treat M1–M6 (isotropic, continuous γ , scalable) as the publishable core; scope later milestones as stretch goals reported as preliminary results if incomplete.

13. Timeline & Resourcing

Assumes one primary researcher with GPU access; adjust to actual team size and compute budget.

Phase	Approx. duration	Exit criterion
P0–P2 (reference distributions, differentiable core, PSD analyzer)	3–4 weeks	Verified nonzero gradients through one full step and through the analyzer
P3–P4 (pairwise energy, force, single- γ training)	3–5 weeks	Loss decreases for $\gamma=+1$; M1 achieved
Section 6.3 baseline comparison	1–2 weeks	Quantified neural-vs-classical comparison at matched γ
P5 (continuous γ)	2–3 weeks	Interpolation to unseen γ verified
P6 (local k-NN energy)	2–3 weeks	Scales to $N \geq 4096$ with preserved spectral quality
P7–P8 (anisotropy, spatial adaptation — stretch)	4–6 weeks	Treated as stretch goals per the risk table
P9 (applications, perceptual study, writeup)	3–4 weeks	Complete experimental story for submission

14. Implementation Plan

Phase	Implementation	Exit criterion
P0	Reference distributions + baseline metrics	Known-distribution ground truth in place

P1	Differentiable particle simulation in PyTorch/JAX	Gradients through one step
P2	Differentiable rasterization + FFT	Correct PSD and verified gradients (finite-difference check)
P3	Pairwise neural energy	E_θ computed correctly
P4	Energy-gradient force	Stable particle motion
P5	End-to-end training	Loss decreases for γ=+1
P6	Continuous γ	Good interpolation/generalization
P7	Local interactions	Scalable O(Nk) model
P8	Anisotropy (stretch)	Accurate 2-D target PSD
P9	Spatial adaptation (stretch)	Local spectral control
P10	Applications + paper	Complete experimental story

Suggested Software Architecture

- `data.py` — initialization distributions and target conditions
- `energy.py` — neural pairwise/local energy
- `dynamics.py` — differentiable time integration (checkpointing/truncated-BPTT switch, Section 6.1)
- `rasterize.py` — differentiable particle-to-grid conversion (Gaussian splat kernel)
- `spectrum.py` — FFT, PSD, radial/2-D analysis
- `losses.py` — spectral, slope, spacing (CV_NND), and anisotropy losses
- `baseline.py` — classical per-target optimizer (Section 6.3)
- `train.py` — training loop
- `evaluate.py` — quantitative evaluation (Section 9.1) and perceptual-study harness (Section 9.2)
- `visualize.py` — distributions, spectra, convergence plots

15. Recommended Initial Hyperparameters

Parameter	Initial value / range
Domain	[0,1] ²
Particles	N=256 initially; later 512–4096+
Target slope γ	γ ∈ [-2, +2]
Integration	Euler or semi-implicit; fixed Δt
Simulation steps	Start with 50–100; tune against Section 6.1 memory budget
Neighbors	k = 16/32/64 after local model
Optimizer	Adam
Primary loss	Log-PSD MSE
Secondary loss	Slope error + CV_NND spatial quality
Training seeds	At least 3 for final comparisons

16. What Not to Do Initially

- Do not assume a hand-designed repulsion/attraction term is required — test the no-stabilizer version first (Section 5, Stage 6).
- Do not begin with Transformers or large diffusion models.
- Do not start with 3-D, anisotropy, and spatial adaptation simultaneously.
- Do not train before validating the PSD measurement pipeline and its gradients.
- Do not rely only on visual examples; report quantitative spectral and spatial metrics.
- Do not assume the neural model beats the classical baseline until Section 6.3’s comparison establishes it.

17. Publication Positioning

A reasonable target hierarchy: SIGGRAPH/ACM TOG first if the method demonstrates broad, technically convincing controllability with the Section 6.3 baseline comparison and Section 9.2 perceptual evidence in hand; Eurographics/Computer Graphics Forum as a strong alternative; EGSR if the main application becomes rendering and sampling.

Potential title: *Learning Spectral Point Distributions with Neural Energy Fields*

18. Central Scientific Claim to Test

A learned energy-based interaction law, with no hand-designed dynamical system to lean on, can produce and continuously interpolate across a family of target point-set spectra — at lower per-target cost than classical optimization — while retaining geometric regularity, controllability, and a provable stability guarantee.

19. Immediate Next Step

Two things in parallel: **(1)** the Section 2 literature pass, sized to roughly a week, to lock in the comparison baseline and novelty framing before writing simulation code; **(2)** a small PyTorch prototype for $N=256$ points in a 2-D square that makes the simulation and PSD differentiable with a verified gradient check, and trains a purely neural pairwise energy (no stabilizing term) for one target $\gamma=+1$. Only after the loss decreases, the generated spectrum moves toward the target, and it's clear whether particle collapse occurs without a stabilizer, should the implementation expand to continuous γ , the Section 6.3 baseline comparison, local interactions, and (if time allows) anisotropy and spatial adaptation.

20. Success Criteria

- **M1:** Differentiable dynamics verified with nonzero, numerically sensible gradients (including the rasterization gradient check).
- **M2:** PSD analyzer reproduces expected behavior of known/reference distributions (Stage 0).
- **M3:** Neural energy learns one target spectrum ($\gamma=+1$), with or without a stabilizing term — record which.
- **M4:** Model learns multiple γ values and interpolates to unseen γ .
- **M5:** Model matches or exceeds the classical per-target baseline (Section 6.3) on spectral/spatial quality at equal γ , and is faster at inference time for new, unseen γ values — both measured, not asserted.
- **M6:** Local neural energy scales to larger point sets ($N \geq 4096$).
- **M7 (stretch):** Anisotropic spectra are reproduced.
- **M8 (stretch):** Spatially varying spectral fields are demonstrated.
- **M9:** At least one graphics application shows a practical advantage, supported by the Section 9.2 perceptual comparison.